

Summary of Lecture on Post-training 2/5/26

Rohan Tolani (rohtol), Archit Kumar (karchit), Maaz Hussain (maazh),
Divya Pothavajhyula (divyabp)

AReaL: A Large-Scale Asynchronous Reinforcement Learning System for Language Reasoning

Problem and Motivation:

In Reinforcement learning for Large Language Models, rewards are given for the entire completed response that an LLM generates. Thus, these models are incentivized to produce better responses, and one effective way to do so is through longer deliberation, which tends to yield higher reward, such as generating more step-by-step reasoning tokens or sampling multiple candidate solutions and selecting the best one. These models, referred to as Large Reasoning Models, have proven to perform better across several domains like math, coding, agentic tasks etc. However, reinforcement learning requires lots of examples so the model can experiment and learn which kinds of responses yield the highest rewards, and these models are prone to generating very long responses on the order of thousands of tokens.

Therefore, in order to run reinforcement learning on a large scale, we need an extremely efficient training system but this task is very challenging for 2 core reasons. Firstly, output lengths of the training model differ greatly, which forces many GPUs to idly while waiting for long sequences to finish because, in a synchronous batch, training cannot begin until every response in the batch is finished. Secondly, reinforcement learning alternates between generating new rollouts and training on these rollouts, which results in idle GPUs because of the constant switching between inference and training workloads. Thus, this paper seeks to resolve these RL training inefficiencies by creating a reinforcement training pipeline that decouples generation from training, thus increasing GPU utilization.

Related Works:

Prior work has shown that RL has become the predominant method for enhancing the reasoning capabilities of LLMs, especially for tasks with well-defined reward functions like mathematics, coding or scientific reasoning. Chain-of-thought reasoning is an underlying factor in these tasks, proving to result in meaningfully better performance. Furthermore, recent research has shown that RL is effective on distilled, open-source language models as well. This research is based on an explicit, defined reward model rather than preference-based RLHF techniques or zero-shot reasoning approaches, and the authors of this paper seek to expand on this defined reward model type of RL.

Furthermore, almost all previous research on reinforcement learning on dense transformer models has focused on running the generation and training phases sequentially on the same hardware. As explained above, this synchronous, sequential approach results in a lot of wasted GPU resources and inefficiency. And while recent literature has proposed techniques like weight resharding to rearrange weights between phases or context switching to swap efficiently between generation and training, this paper instead minimizes these wasted

resources through asynchronous reinforcement learning that decouples generation from training.

Finally, prior research on asynchronous reinforcement learning, where data is generated and the model is trained simultaneously, has been successful in several domains like the gaming industry. Systems like AlphaStar, DeepMind's AI that learned to play StarCraft II, were trained by running copies of the agent in parallel to play games and continuously feeding game outcomes back to a learner for model updates. However, this paper builds upon such asynchronous reinforcement learning literature, by applying it to large language models with large context settings, where generation is of variable length and computationally expensive.

Solution Overview:

This paper proposes AREAL, an asynchronous reinforcement learning system designed to address the inefficiencies with synchronous reinforcement learning stated above. The core idea here is to break away from the rigid loop of generating responses and then training, where one stage needs to finish before the other can start again. Instead, AREAL runs LLM generation such that each rollout worker generates new outputs without waiting for the rest of the batch to finish, while the trainer workers run parallel model updates whenever they receive a full training batch collected from the rollout workers.

Because rollout workers continuously generate new outputs without having to wait for every sequence in the batch to finish generating, AREAL largely eliminates the idle time caused by the variable output token lengths. At the same time, AREAL addresses the idle GPUs caused by constantly switching between inference and training workloads by running generation and training simultaneously in separate workers. This design allows rollout workers to stream token generation continuously, while trainer workers update the model as soon as enough data is available, so generation and training can proceed in parallel instead of taking turns.

The tradeoff to an asynchronous RL training pipeline is that, because generation and training is no longer sequential, each training batch contains data from multiple prior policy versions which breaks the PPO assumption that all actions are generated from a single policy and has empirically been proven to degrade model performance. The paper addresses this staleness by introducing parameter η , representing the maximum allowed staleness in each training batch, and uses a rate-limiting protocol that prevents new generation requests whenever the rollout data becomes too outdated. By doing this, the system ensures that there is not an excessive amount of policy mismatch that could significantly degrade performance while also increasing efficiency of the RL training system.

Limitations:

Despite the efficiency and scalability gains demonstrated by AREAL, the system has several limitations. First, the ratio between rollout (inference) workers and trainer workers is manually configured and remains fixed throughout training. This static allocation may be suboptimal across different training phases, model sizes, or task distributions. In particular, as reasoning-oriented fine-tuning often leads to progressively longer contexts and outputs, a fixed inference–training ratio can increase rollout staleness or underutilize available compute resources.

Second, AREAL relies on staleness-aware mechanisms to mitigate the policy mismatch introduced by asynchronous PPO. While effective in practice, this approach introduces additional system complexity and requires careful tuning of the staleness threshold. Improper configuration may still lead to degraded learning stability or reduced sample efficiency, and the need for manual tuning limits ease of deployment.

Third, the evaluation primarily focuses on single-step reasoning tasks with well-defined reward functions, such as mathematics and coding. These settings do not fully reflect the challenges of more interactive or long-horizon environments, where state evolves over multiple turns and rewards may be sparse or delayed. As a result, the reported performance improvements may not directly transfer to more complex agentic scenarios. Finally, AREAL assumes access to large-scale distributed compute infrastructure and significant engineering effort, which may limit accessibility for smaller research groups.

Future Research Directions:

Several directions remain open for extending and improving AREAL. One promising avenue is the development of dynamic resource allocation strategies that adapt the ratio between inference and training workers during training. Such mechanisms could respond to changes in context length, rollout throughput, or learning progress to further improve efficiency and reduce policy staleness.

Another important direction is exploring reinforcement learning objectives and optimization algorithms that are inherently more robust to off-policy data. This could reduce reliance on strict staleness controls and simplify asynchronous training, enabling greater flexibility in system design.

Extending AREAL to multi-turn interactions and agentic settings is also a key area for future research. Applying asynchronous reinforcement learning to environments involving tool use, long-horizon planning, or interactive feedback would help evaluate the generality of the system beyond single-step reasoning tasks.

Finally, future work could focus on improving the accessibility of AREAL by developing lighter-weight implementations or abstractions that reduce infrastructure requirements, making asynchronous RL training more broadly usable across different research settings.

Summary of Class Discussion:

The class discussion was mostly towards the Deepseek side, but also touched upon AREAL. The AREAL discussion clarified a bit more about base PPO as well went a bit more in depth into theory about their Decoupled PPO algorithm. We discussed further upon the rationale behind the change, which was mainly to handle the assumption in standard PPO that there is only ever 1 old model to iterate on for the importance ratio. In AREAL the policies are paralyzed for efficiency, but this breaks that assumption. So the authors decoupled the policy and included the proximal policy that is used in the importance ratio. We did not cover their proof that this decoupling is still correct, but the speakers noted where in the paper that could be found. We also talked about picking staleness constants, and compared the paper's tests in order to come up with a reasonable value. However we also talked about how this is really dependent on the model. This led to a discussion more on the industry side of things and we discussed the practical limitations of using this in industry, mainly how it requires a lot of experimentation to

find the best parameters. Finally, we also talked about how future and current scalable AI is done not only cross machine, but also cross-data center, which possesses a lot of challenges, including idle systems as data is transferred cross-data center.

Questions and Answers:

Q: Can student teacher models provide some more training efficiency?

A: Teacher student models, also called distillations, allows you to make smaller, easier to run models, but you still need to use the big models to train it so you are training the big model nevertheless.

Q: How do you do staleness optimization (pick the staleness parameter)?

A: Its very problem set is dependent (so should be trained for each problem and model), but in the paper's cases they found a staleness of about 2 to be empirically decent.

Follow up: Also the scale of the problem affects how well you can tune staleness, so it's hard to tell theoretically what is a good stateless for your problem; it all has to be done empirically.

In industry, you can't run it a bunch of times to find the best staleness, which is a partial limitation of stale-synchronous systems like AREAL. Sometimes you can do a small training and project, but that only works if you have a good prior idea of the model and its loss curve.

Q: PPO ratio

A: In standard PPO the importance ratio assumes theres 1 old model, but in AREAL that's not the case so the solution was this decoupling, instead of having 1 old policy, there's a heavier one to generate the batch, and the proximal policies helps with clipping and regularization since it is more updated

Q: Asynchronous vs synchronous usage

A: Very interesting topic in industry, for example a model called "PAPAYA: Practical, Private, and Scalable Federated Learning" was once SOTA and in that paper they implement asynchronous training in order to address waiting nodes. However a few years later PAPAYA authors published more research partially saying that it wasn't that effective anymore.

DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning

Problem and Motivation:

Recent progress in LLMs has shown that they can perform complex reasoning tasks such as mathematical problem solving, logical deduction, and complex programming. A major driver of this has been chain-of-thought (CoT) reasoning, where models generate intermediate reasoning steps before producing a final answer. CoT significantly improves performance on many reasoning benchmarks. However, current CoT based approaches rely heavily on human-annotated reasoning traces, through prompting or SFT. These annotations are expensive to collect, difficult to scale, and may encode human biases or suboptimal reasoning strategies, potentially limiting a model's performance and its ability to explore alternative reasoning paths.

Prior reinforcement learning approaches to improving LLM reasoning often still depend on SFT or learned reward models that score intermediate reasoning steps. These reward models are complex, prone to reward hacking, and unreliable for long reasoning trajectories. As a result, it remains unclear whether strong reasoning behavior could be learned without explicit human guidance on how to reason.

DeepSeek-R1 addresses this gap by investigating whether reasoning capabilities can emerge purely from reinforcement learning, using only outcome-based feedback that evaluates the correctness of final answers. The motivation of this work is to reduce reliance on human-labeled reasoning data and enable models to autonomously develop effective reasoning strategies (e.g. self-verification or reflection) through self-exploration. Successfully doing so would provide a more scalable and powerful path toward improving reasoning in LLMs.

Related Works:

This paper builds upon many lines of research. First, scaling laws and emergent abilities show that reasoning-like behavior can be developed in larger models, but at a large computational cost during pretraining (Kaplan et al., 2020; Wei et al., 2022a). Second, chain-of-thought prompting demonstrates that explicit examples of intermediate reasoning steps leads to much better performance on reasoning benchmarks, but is fully reliant on human prompts and examples (Wei et al., 2022b; Kojima et al., 2022).

In more recent research, applying SFT on long-form reasoning traces or RL with learned reward models has shown to be effective; however, similar to earlier, these methods are heavily reliant on human annotations or reward models that score the reasoning quality itself. This makes it hard to scale and is also prone to reward hacking.

DeepSeek-R1 builds upon these lines of research by first completing eliminating supervision over intermediate reasoning. Instead of teaching a model how to reason, it focuses on incentivizing the reasoning process using pure RL with rule-based and outcome-only rewards, which in turn has shown to allow reasoning behavior and strategies to emerge through exploration.

Solution Overview:

DeepSeek-R1 introduces a training framework that incentivizes reasoning in LLMs using RL with outcome-based rewards, rather than supervised learning on human-annotated reasoning traces. The approach starts with a large pretrained model (DeepSeek-V3-Base) and applies RL where the reward only depends on the correctness of the model's final answer, without explicitly constraining how the reasoning is produced.

The core optimization method is Group Relative Policy Optimization (GRPO), an efficient variant of PPO that avoids training a separate value model. For each prompt, the model samples a group of candidate responses, which are scored using rule-based verifiers such as exact answer matching for math questions, or unit-test execution for coding tasks. Each response is evaluated relative to others in the same group, encouraging the model to favor better-than-average reasoning trajectories.

Using this setup, the authors first train DeepSeek-R1-Zero, a model optimized purely through reinforcement learning. Despite receiving no supervision on intermediate reasoning steps, R1-Zero naturally developed long CoT behaviors such as self-verification, reflection, and exploration of alternative solution paths. This leads to large performance gains on verifiable reasoning tasks, particularly in math, coding, and STEM benchmarks. However, R1-Zero exhibits practical issues such as poor readability, language mixing, and weak performance on non-reasoning tasks.

To address these limitations, the authors introduce a multi-stage post-processing pipeline to produce DeepSeek-R1. This pipeline incorporates a small amount of SFT using human-aligned “cold-start” reasoning data, in order to improve readability and language consistency. Next, rejection sampling is done to keep high-quality reasoning outputs. A second RL stage is then applied, combining rule-based rewards for reasoning tasks with preference-based reward models that optimize helpfulness and safety on general instruction-following tasks. This final stage allows DeepSeek-R1 to keep the strong reasoning capabilities learned through pure RL while improving its alignment, usability, and general performance.

The key insight this paper demonstrates is that pretrained LLMs already contain reasoning capabilities and that outcome-based RL is enough to bring out these capabilities without requiring human-labeled examples of reasoning traces.

Limitations:

Although DeepSeek-R1 achieves SOTA performance on many reasoning benchmarks, it still has several important limitations. First, the model's structured output and tool use capabilities are limited. DeepSeek-R1 cannot currently use external tools such as calculators, search engines, or compilers during inference, which restricts its performance on tasks where tool-assisted reasoning would be beneficial.

Second, the model is not fully token-efficient. While DeepSeek-R1 dynamically allocates more reasoning to harder problems and fewer tokens to simpler ones, it still sometimes overthinks, producing unnecessarily long CoT outputs for relatively easy questions. This leads to higher inference costs than necessary.

Third, DeepSeek-R1 is mainly optimized for English and Chinese, which can cause language-mixing issues when handling prompts in other languages. In some cases, the model

may reason/respond in English when the query is written in different languages. This limitation is likely inherited from the base model, which was also primarily trained on English and Chinese data.

The model also shows sensitivity to prompting strategies. In particular, few-shot prompting consistently hurts performance. DeepSeek-R1 performs best in zero-shot settings where the problem is clearly described and the expected output format is specified.

Finally, DeepSeek-R1 has not shown large improvements on software engineering tasks. Due to the long evaluation times required for these tasks, large scale RL was not extensively applied in this domain, limiting gains.

Beyond these task-specific issues, the pure RL approach introduces challenges. In particular, pure RL relies on reliable reward signals. While rule-based rewards work well for verifiable tasks like math and coding, they are difficult to design for open-ended tasks like writing. When reward signals are provided by learned reward models instead of fixed rules, the system is more vulnerable to reward hacking, where the model exploits weaknesses in the reward function rather than actually improving performance.

Future Research Directions:

This work opens several promising avenues for future research. One important direction is the development of tool-augmented reasoning, where RL environments allow models to interact with tools such as calculators, search engines, or domain-specific systems (e.g. OpenHands). This would significantly expand the set of tasks that can be effectively trained using outcome-based RL.

Improving token efficiency is another key direction. Future models could incorporate cost-aware reward signals that penalize unneeded reasoning, encouraging more efficient use of computation while preserving accuracy on difficult problems.

Addressing the limitations of reward design is also important. Developing more robust and reliable reward models for tasks that are not easily verifiable (e.g. writing tasks) would allow pure RL methods to scale to a wider range of applications without heavily relying on human annotation.

Finally, extending this language to additional languages, domains, and modalities, as well as continuing to explore model distillation would help validate the generality of DeepSeek-R1 and make advanced reasoning models more practical and usable.

Summary of Class Discussion:

Class discussion focused on why RL was essential to DeepSeek-R1's reasoning capabilities and how it differs from SFT. Unlike SFT, which trains models to imitate human-written reasoning and is limited by the availability and quality of annotated data, RL allows the model to learn *how* to arrive at the correct answer. Because RL only depends on a verifier, it scales more easily and allows reasoning behaviors to emerge organically. This was seen as a key reason to why R1 was able to develop verification and reflection without access to CoT examples.

The class also discussed behaviors of the model, such as language mixing, few-shot sensitivity, and overthinking. Forcing the model to reason in a single language improved interpretability, even though allowing multiple languages might lead to more powerful internal

reasoning. Few-shot prompting was noted to hurt performance because the model was not trained with few-shot CoT examples and instead learned reasoning from no base. The tendency to overthink was attributed to the training pipeline itself. The model was optimized on reasoning-type prompts early on, encouraging longer reasoning chains. Non-reasoning data was introduced only at the final stage. Overall, the discussion highlighted that DeepSeek-R1's reasoning depth and token usage are learned behaviors, rather than being explicitly taught.

Questions and Answers:

Q: What did using RL add to the training process (versus other techniques)?

A: RL allows the model to figure out how to get to the right answer, rather than just getting the correct answer. RL needs a lot of data, but it does not necessarily need to be human generated. SFT is not as scalable since it requires human-annotated data, its reasoning capability is capped, and has shown to develop more as imitation over reasoning.

Q: Does the model need to be forced to reason in one language; why can we not use multiple languages?

A: Forcing the model to reason in one language allows much better human interpretability. The authors do note that potentially using multiple languages can lead to a more powerful model; however, this comes at the cost of being more human readable.

Q: Why does few-shot decrease the performance of the model?

A: Few-shot does degrade performance, and the intuition behind why is that there was no chain-of-thought data from which the model could learn from, which forces the model to learn how to reason from no base.

Q: Why does the DeepSeek model tend to 'overthink'?

A: Partly due to the RL pipeline since the model is being trained on the reasoning-type prompts, which encourages more thinking to get to the final answer. The non-reasoning prompts are only provided to the model at the last stage, at which point the model has already learned more in-depth reasoning behaviors.

Q: How does the model learn to reason?

A: The model is not provided with any chain-of-thought reasoning examples. This behavior is a result of the reward model which is accuracy plus format, which allows the model's reasoning behavior to emerge over time since a correct answer is more likely after more reasoning (for reasoning-type prompts).

Q: How does Deepseek-R1 determine how many tokens to generate?

A: The model learns from training. However, it's difficult to get the model to produce a certain amount of tokens.